

LAB IMPLEMENTATION 4

Blockchain Web Application Using Solidity and MetaMask

BC

Aim

To develop a blockchain-powered web application using Solidity on Remix IDE and integrate a MetaMask wallet for smart contract deployment and transactions on a test network.

Theory

A blockchain web application can use a **smart contract** to store and process data on a blockchain network. Solidity is used to define the contract logic, while Remix IDE provides facilities for writing, compiling, deploying, and interacting with Solidity contracts. MetaMask acts as a wallet and browser interface for approving blockchain transactions from a user account.

A **testnet** is a blockchain network intended for development and testing without using the main production network. In this experiment, Remix is connected through the browser wallet to the Sepolia testnet. The `SimpleStorage` contract maintains a public unsigned integer and provides a state-changing `setValue` function and a read-only `getValue` function. Deployment and state-changing calls require transaction approval through MetaMask.

Steps

1. Install and open MetaMask and create or import a wallet account.
2. Select the Sepolia testnet and provide test Ether to the wallet.
3. Open Remix IDE and create the Solidity file `SimpleStorage.sol`.
4. Enter the `SimpleStorage` contract and compile it with a compatible Solidity compiler.
5. Select `Browser Extension` as the Remix deployment environment and connect the MetaMask account.
6. Select the Sepolia testnet and review the deployment request in MetaMask.
7. Confirm the transaction in MetaMask and deploy the smart contract.
8. Inspect the deployed contract and transaction information shown by Remix and the explorer link.
9. Interact with `setValue` to submit a value and use `getValue` to read the stored value.

Code

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract SimpleStorage {
5     uint256 public value;
```

```
6
7     function setValue(uint256 _value) public {
8         value = _value;
9     }
10
11     function getValue() public view returns (uint256) {
12         return value;
13     }
14 }
```

Listing 1. SimpleStorage.sol

Output

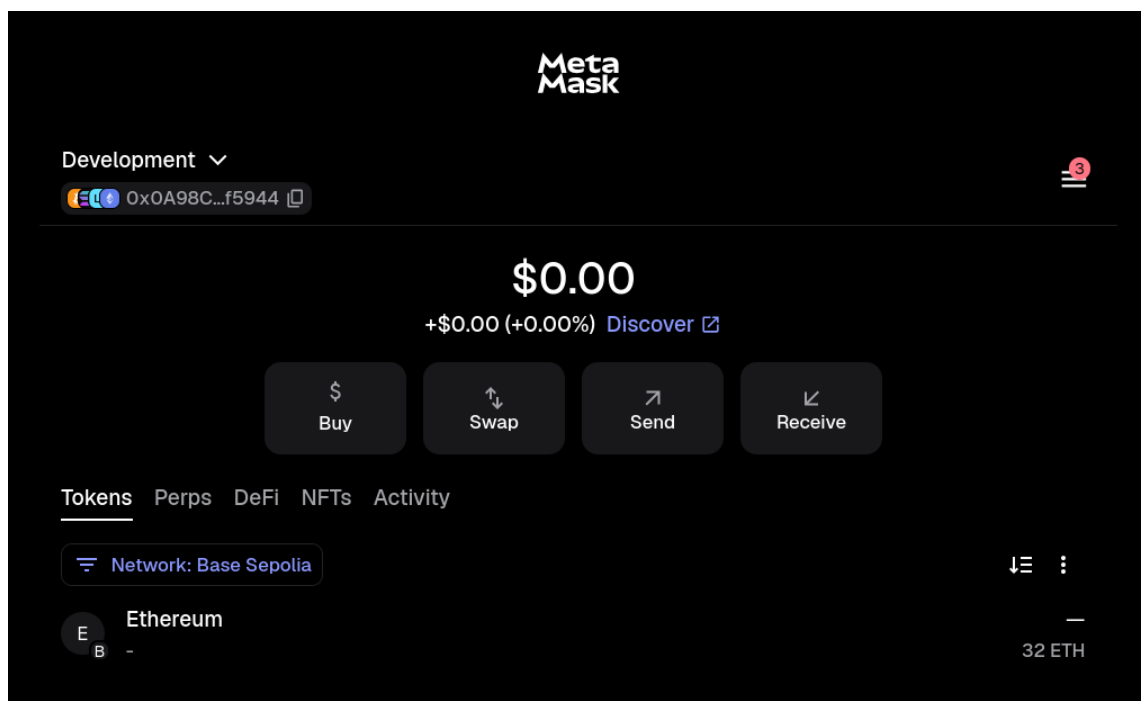


Fig. 1. MetaMask wallet connected to the Base Sepolia network

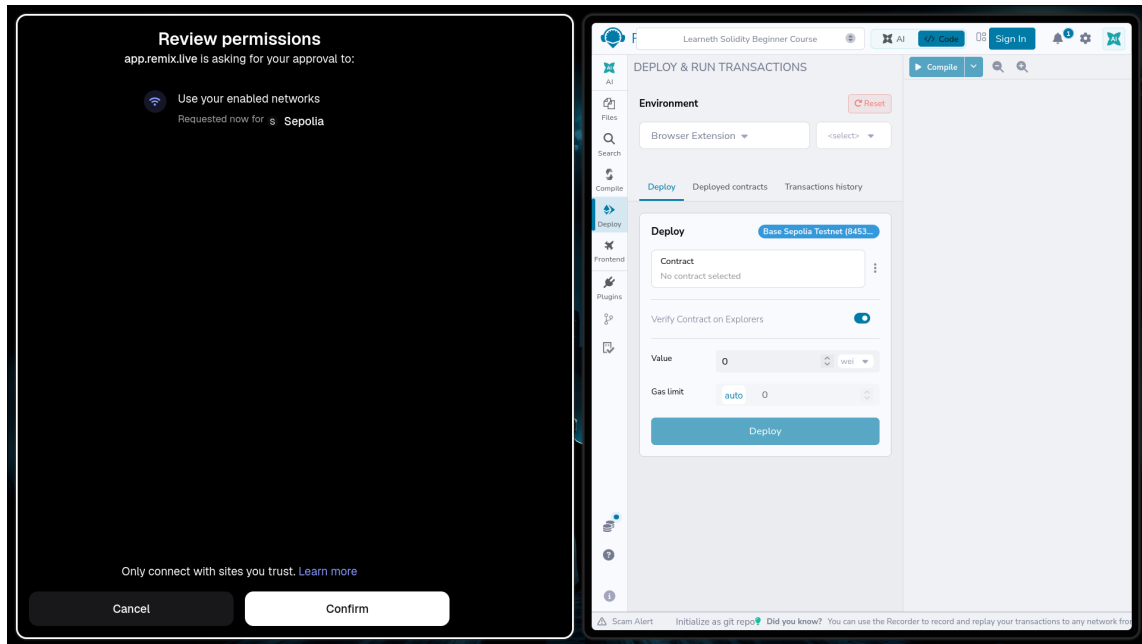


Fig. 2. MetaMask permission request and Remix deployment interface

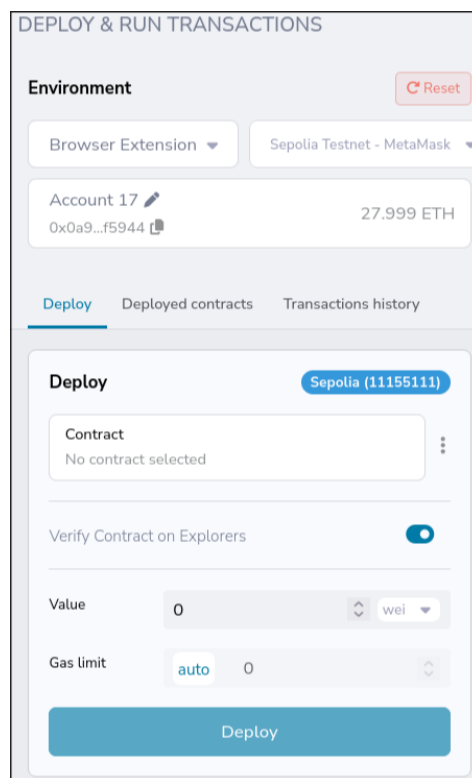
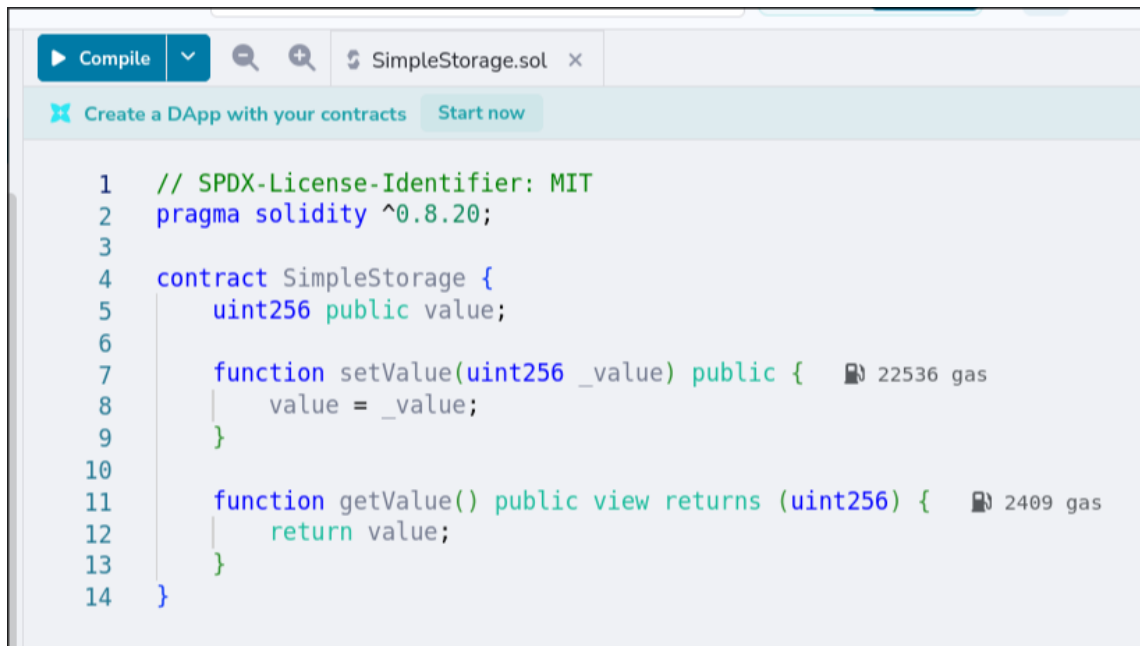


Fig. 3. Remix deployment panel configured for the Sepolia testnet



```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract SimpleStorage {
5     uint256 public value;
6
7     function setValue(uint256 _value) public { 22536 gas
8         value = _value;
9     }
10
11     function getValue() public view returns (uint256) { 2409 gas
12         return value;
13     }
14 }
```

Fig. 4. SimpleStorage contract source code in Remix IDE

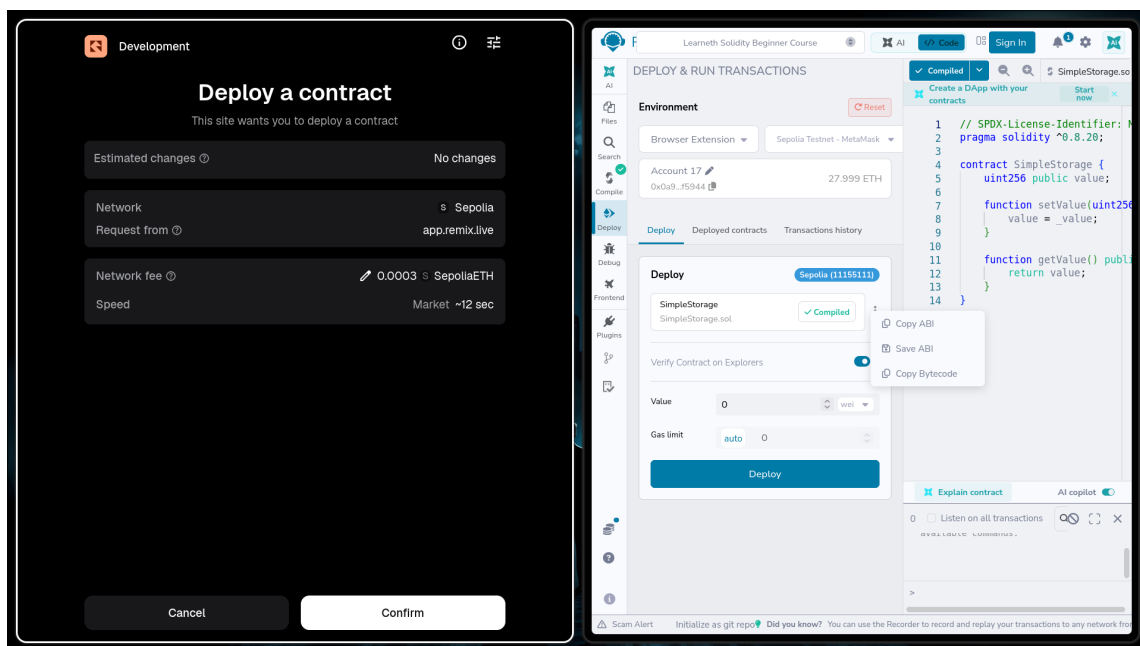


Fig. 5. MetaMask confirmation for deploying the smart contract

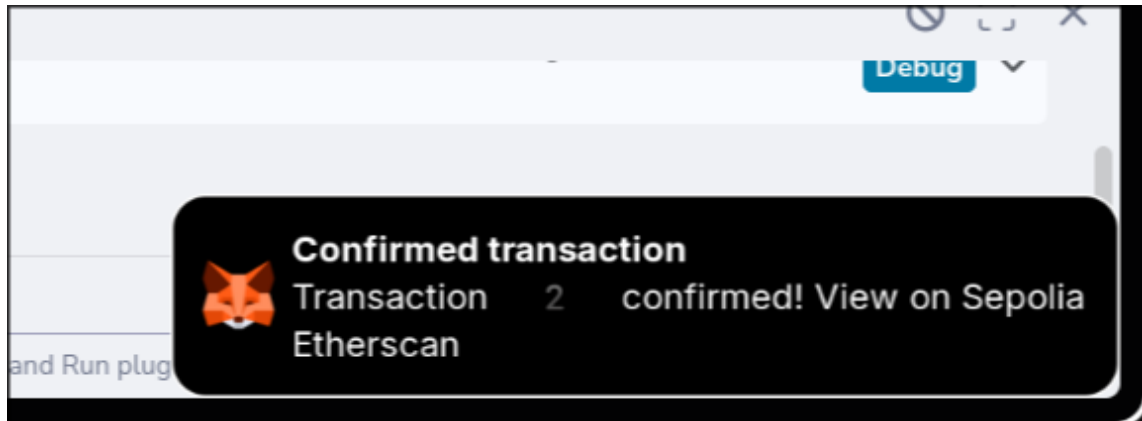


Fig. 6. Deployed contract and confirmed transaction details in Remix

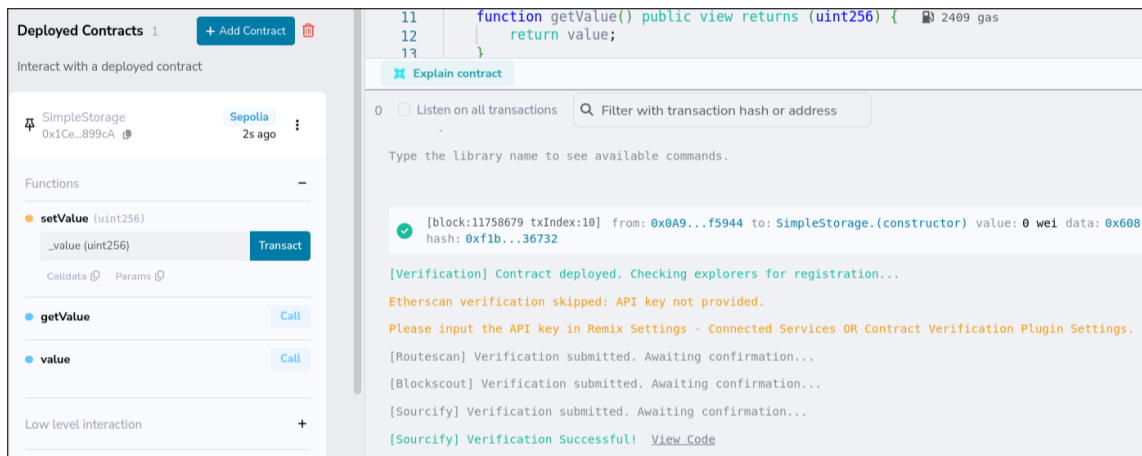


Fig. 7. MetaMask transaction request for a contract interaction

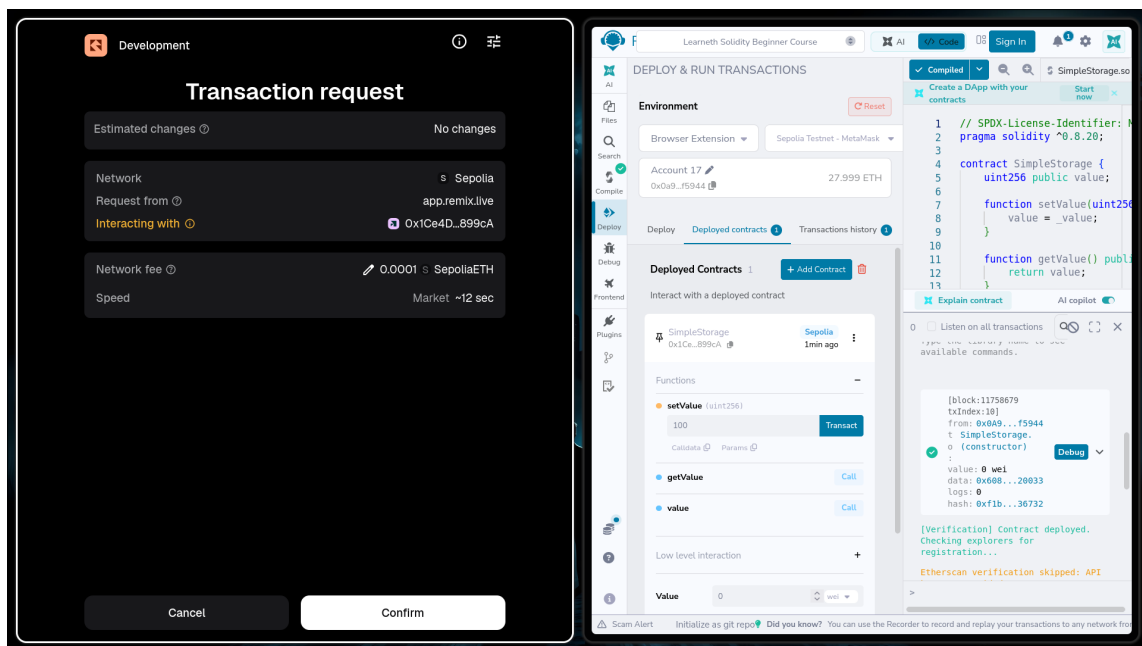


Fig. 8. Remix interface showing the deployed contract interaction

Conclusion

The `SimpleStorage` smart contract was prepared in Solidity and used with Remix IDE. The supplied screenshots showed MetaMask connected to a test network, Remix configured for browser-extension deployment, deployment confirmation, confirmed transaction information, and contract interaction. The experiment demonstrated the workflow for connecting MetaMask with Remix and performing smart contract transactions on a testnet.